SENAI AFONSO GRECO

CURSO TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS

Autor: Otavio

Professor: Frederico Martins Aguiar

Tema: Implementação do Padrão Estrutural *Facade*

RELATÓRIO TÉCNICO: APLICAÇÃO DE PADRÕES DE PROJETO NO DESENVOLVIMENTO C# WPF

NOVA LIMA - MG 2026

OTAVIO HENRIQUE BARBOSA SOARES

RELATÓRIO TÉCNICO: APLICAÇÃO DO PADRÃO DE PROJETO FACADE EM C# WPF

Relatório técnico apresentado à disciplina de Desenvolvimento de Sistemas do curso de Desenvolvimento de sistemas do Senai Afonso Greco, como requisito parcial para obtenção de nota na Situação de Aprendizagem.

Professor: Frederico Martins Aguiar

NOVA LIMA – MG 2026

1. Introdução e Contextualização do Problema O desenvolvimento de software corporativo exige arquiteturas que suportem escalabilidade e manutenção contínua. A situação-problema apresentada descreve um cenário crítico comum na engenharia de software: sistemas com alto acoplamento, baixa coesão e dificuldade de adaptação a novos requisitos. Em aplicações Desktop desenvolvidas com Windows Presentation Foundation (WPF) e C#, a ausência de padrões arquiteturais frequentemente resulta no encapsulamento de regras de negócio complexas diretamente no *code-behind* das interfaces gráficas (Views).

Para solucionar essa deficiência arquitetural, este projeto propõe a implementação do padrão de projeto *Facade* (GoF). O objetivo é demonstrar como a introdução de uma camada intermediária pode isolar a interface do usuário das complexidades dos subsistemas de *back-end*, promovendo a separação de responsabilidades (Separation of Concerns).

2. Fundamentação Teórica: O Padrão Facade Definido pelo *Gang of Four* (Gamma et al., 1994) no catálogo de padrões estruturais, o *Facade* tem a intenção de "fornecer uma interface unificada para um conjunto de interfaces em um subsistema. O Facade define uma interface de nível mais alto que torna o subsistema mais fácil de ser usado".

A estrutura do padrão é composta pelos seguintes participantes:

- **Facade:** Conhece quais classes do subsistema são responsáveis por atender a uma requisição e delega as solicitações do cliente aos objetos apropriados.
- **Subsistemas:** Implementam a funcionalidade real do sistema. Lidam com o trabalho atribuído pelo objeto Facade e não possuem conhecimento ou referência à Fachada.

- **Client:** A aplicação ou módulo (neste caso, a interface WPF) que interage exclusivamente com a Fachada.

3. **Justificativa da Escolha Arquitetural** A escolha do padrão *Facade* fundamenta-se na necessidade direta de reduzir o acoplamento descrito no problema da empresa. Ao desenvolver uma tela de "Checkout de Vendas" em WPF, ações como validação de estoque, processamento de pagamento e disparo de notificações são exigidas simultaneamente. Se a classe `MainWindow.xaml.cs` instanciar e gerenciar todos esses serviços, ocorrerá uma forte violação do Princípio da Responsabilidade Única (SRP - *Single Responsibility Principle*) do SOLID (Martin, 2008). O *Facade* atua como um orquestrador, permitindo que a View dependa de uma única abstração.

4. **Implementação Prática no Projeto WPF** A aplicação foi estruturada separando a lógica de negócios da interface. Foram criados três subsistemas independentes: `EstoqueService`, `PagamentoService` e `NotificacaoService`. Em seguida, implementou-se a classe `VendaFacade`, responsável por instanciar esses serviços e expor um método público `FinalizarPedido()`.

Na camada de apresentação (WPF), a interface foi aprimorada para coletar dados reais do usuário: agora, além de clicar no botão para finalizar o pedido, é possível digitar o valor da compra, o número do cartão de crédito e o e-mail do cliente. Com isso, o evento do botão de finalização de compra foi reduzido a apenas instanciar a `VendaFacade`, passar os parâmetros coletados dos controles (*TextBoxes*) e aguardar o retorno em *string*, que é exibido no status da tela. Nenhuma regra de negócio é processada na camada visual.

5. **Análise Crítica e Comparativa** A abordagem tradicional (sem o padrão) resultaria em um código procedural e frágil. Qualquer alteração na API de pagamentos ou na lógica de notificação exigiria a recompilação e alteração da interface gráfica.

Com a implementação do *Facade*, observam-se as seguintes **vantagens**:

- **Desacoplamento:** A interface gráfica desconhece a existência de regras de pagamento ou estoque.

- **Manutenibilidade:** Alterações no fluxo de vendas são feitas centralmente na Fachada.

- **Testabilidade:** Torna-se possível isolar a Fachada para testes unitários, zombando (mocking) os subsistemas.

Como **desvantagem**, ressalta-se o risco de a Fachada se tornar um *God Object* (Objeto Deus) ou um *Blob*, centralizando lógicas excessivas caso o sistema cresça descontroladamente sem a devida subdivisão de múltiplas fachadas por contexto de negócio (Freeman C Robson, 2004).

6. Aplicações no Mercado Atual O padrão é exaustivamente utilizado no mercado em:

- **Gateways de Pagamento:** APIs como Stripe ou PayPal oferecem bibliotecas de Fachada onde o desenvolvedor chama apenas `CreateCharge()`, ocultando a complexidade de criptografia e comunicação bancária.

- **Frameworks de Persistência:** O `DbContext` do Entity Framework Core em C# atua como uma fachada sobre o ADO.NET, gerenciando conexões, transações e mapeamento objeto-relacional de forma transparente para o desenvolvedor.

7. Conclusão A adoção do *Facade* provou ser a solução arquitetural ideal para o problema de alto acoplamento apresentado. A aplicação WPF desenvolvida demonstra de forma clara que a separação entre UI e lógica de negócios não apenas torna o código legível, mas garante que o sistema seja facilmente extensível, atendendo aos padrões de qualidade da engenharia de software moderna.

Referências Bibliográficas

- GAMMA, Erich; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. *Padrões de Projeto: Soluções Reutilizáveis de Software Orientado a Objetos*. 1. ed. Porto Alegre: Bookman, 2000.

- FREEMAN, Eric; ROBSON, Elisabeth. *Use a Cabeça! Padrões de Projetos*. 2. ed. Rio de Janeiro: Alta Books, 2007.

- MARTIN, Robert C. *Código Limpo: Habilidades Práticas do Agile Software*. 1. ed. Rio de Janeiro: Alta Books, 2011.

Link Repositorio do GIT https://github.com/OtavioHub97/sa-design-patterns-wpf.git